

CODE WALKTHROUGH

Stock Performance Regression Analysis

A Plain-English, Line-by-Line Guide to the Python Code

This document explains every line of code used in the analysis — in plain English, with no prior Python experience required. Each code block is shown exactly as written, followed by a clear explanation of what it does and why. Real-world analogies are used throughout to make statistical concepts easy to grasp.

Project: stock-performance-regression-analysis | June 2026

1 Importing the Tools (Libraries)

Before doing any analysis, Python needs to be told which toolkits to load. Think of this like opening a toolbox before starting a job — you're picking which tools you'll need.

Loading the libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
pd.options.display.float_format = "{:, .2f}".format
```

- ✓ `import pandas as pd` — loads Pandas, the main tool for working with tables of data (like Excel in Python). We nickname it 'pd' to save typing.
- ✓ `import numpy as np` — loads NumPy, which handles mathematical calculations. Nicknamed 'np'.
- ✓ `import matplotlib.pyplot as plt` — loads the chart-drawing tool. Nicknamed 'plt'.
- ✓ `import seaborn as sns` — loads Seaborn, a prettier chart library built on top of Matplotlib. Nicknamed 'sns'.
- ✓ `import statsmodels.formula.api as smf` — loads the statistical modelling library we use to run linear regression. Nicknamed 'smf'.
- ✓ `pd.options.display.float_format = ...` — tells Pandas to display all numbers with 2 decimal places and a comma separator (e.g. 1,234.56). Just a cosmetic setting.

■ *Analogy: Like gathering your calculator, ruler, graph paper, and textbooks before starting a maths exam.*

2 Loading & Inspecting the Stock Price Data

Now we load the stock price spreadsheet into Python and take a first look at it — checking its shape, contents, and whether any data is missing.

Loading the CSV file

```
prices = pd.read_csv("stock_prices.csv", index_col = "Date", parse_dates = ["Date"])  
prices
```

- ✓ `pd.read_csv("stock_prices.csv")` — reads the CSV file from disk into a table (called a DataFrame).
- ✓ `index_col = "Date"` — tells Pandas to use the Date column as the row label (the index), not a regular column.
- ✓ `parse_dates = ["Date"]` — tells Pandas to recognise the Date column as actual dates, not plain text.
- ✓ `prices` — typing the variable name at the end of a cell displays the table in Jupyter.

■ *Analogy: Like opening a spreadsheet in Excel and seeing rows for each date and columns for each stock.*

Previewing the first rows

```
prices.head
```

- ✓ `prices.head` — shows the first 5 rows of the table so you can quickly confirm the data looks correct.
- ✓ This is one of the first sanity checks you do whenever you load a new dataset.

Checking for missing values

```
prices.isnull().sum()
```

- ✓ `prices.isnull()` — creates a True/False table — True wherever a value is missing (NaN), False where data exists.
- ✓ `.sum()` — counts the number of True values per column, giving you the total missing count per stock.
- ✓ If all values are 0, the dataset is complete — no gaps.

■ *Analogy: Like checking an attendance register to see if anyone is absent.*

Inspecting data types and memory

```
prices.info()
```

- ✓ `prices.info()` — prints a summary showing: number of rows, column names, how many non-null (non-missing) values each column has, and the data type of each column.
- ✓ We want to confirm all 15 stock columns are float64 (decimal numbers) and the index is a DatetimeIndex (real dates, not text).

Saving the list of stock names

```
symbols = prices.columns.tolist()
```

- ✓ `prices.columns` — gives us all column names (the 15 stock tickers).
- ✓ `.tolist()` — converts them from a Pandas Index object into a plain Python list.
- ✓ `symbols` — we save this list so we can loop through the 15 stocks later without typing each name manually.

■ *Analogy: Writing down the names of 15 students so you can call them one by one later.*

3 Converting Prices to Returns (Stationarity)

Raw stock prices are not suitable for linear regression because they trend upward over time (non-stationary). We convert them into daily percentage returns, which fluctuate around a stable mean — a property called covariance stationarity. This is a fundamental requirement for valid time series regression.

Plotting raw stock prices

```
prices.plot(figsize = (12, 8))  
plt.show()
```

- ✓ `prices.plot(figsize = (12, 8))` — draws a line chart of all 15 stock prices over time. `figsize` sets the chart width (12 inches) and height (8 inches).
- ✓ `plt.show()` — tells Python to display the chart on screen.
- ✓ Purpose: you will see prices trending upward — this upward drift is the non-stationarity we need to remove.

■ *Analogy: Looking at a house price history chart — prices only go up over time, which makes them hard to model statistically.*

Calculating daily percentage returns

```
returns = prices.pct_change().mul(100).dropna()
```

- ✓ `prices.pct_change()` — calculates the percentage change from each day to the next, for every stock. Formula: $(\text{today} - \text{yesterday}) / \text{yesterday}$.
- ✓ `.mul(100)` — multiplies by 100 to express returns as a percentage (e.g. 0.5 means +0.5%, not 0.005).
- ✓ `.dropna()` — removes the first row, which becomes NaN because there is no 'previous day' for the very first date.
- ✓ `returns` — the resulting DataFrame now contains daily return percentages for all 15 stocks.

■ *Analogy: Instead of tracking a stock's price (\$150, \$153, \$149...), you track its daily moves (+2%, -2.6%...). The moves stay within a consistent range — that's stationarity.*

Plotting returns to confirm stationarity

```
returns.plot(figsize = (12, 8))  
plt.show()
```

- ✓ Same as before — but now we plot returns instead of prices.
- ✓ The chart should show lines oscillating randomly around zero, with no upward or downward trend.
- ✓ This visual confirmation tells us the time series is now stationary and suitable for regression.

4 Loading & Inspecting the Fama/French Factors

Now we load the second dataset — the Fama/French 5 Factors. These are the independent variables (X) in our regression models. They are already expressed as daily returns (%), so no transformation is needed.

Loading the factors CSV

```
factors = pd.read_csv("factors.csv", index_col = "Date", parse_dates = ["Date"])  
factors
```

- ✓ Same approach as stock prices — `pd.read_csv()` reads the file, `index_col="Date"` sets dates as the row index, `parse_dates=["Date"]` ensures dates are recognised properly.
- ✓ `factors` — displays the table. Columns should be: `Mkt_Prem`, `SMB`, `HML`, `RMW`, `CMA`, `RF`.
- ✓ `RF` is the daily risk-free rate (1-month US Treasury bill return) — used later to compute excess returns.

Checking factors info and missing values

```
factors.info()
```

- ✓ Same as before — confirms no missing values and that all columns are numeric (float64).
- ✓ We also confirm the date index aligns with the stock returns dataset.

Plotting the five factors

```
factors.drop(columns = ["RF"]).plot(figsize = (12, 8))  
plt.show()
```

- ✓ `factors.drop(columns = ["RF"])` — temporarily removes the `RF` column from the plot, since `RF` is the risk-free rate (not a factor we model with).
- ✓ `.plot(figsize = (12, 8))` — plots the five factors as line charts over time.
- ✓ Visual check: the factors should appear stationary (fluctuating around zero, no trends) — confirming they are ready for regression.

Saving the five factor names as a list

```
fivefactors = factors.columns[:5].to_list()
```

- ✓ `factors.columns[:5]` — takes the first 5 column names (`Mkt_Prem`, `SMB`, `HML`, `RMW`, `CMA`), excluding `RF`.
- ✓ `.to_list()` — converts to a Python list.
- ✓ `fivefactors` — we save this list to use later when building regression formulas and filtering data.

5 Merging Datasets & Computing Excess Returns

We now combine the stock returns and factors into a single table, then compute the excess return for each stock — the stock's return minus the risk-free rate. This excess return is what each regression model will try to explain.

Merging the two DataFrames

```
data = pd.concat([returns, factors], join = "outer", axis = 1)
```

- ✓ `pd.concat([returns, factors])` — stacks the two DataFrames side by side (horizontally).
- ✓ `join = "outer"` — keeps all dates from both datasets, even if one is missing a date the other has (gaps are filled with NaN).
- ✓ `axis = 1` — means we join along columns (side by side), not rows (stacked top to bottom).
- ✓ `data` — the resulting combined DataFrame has all 15 stock return columns plus all 6 factor columns.

■ *Analogy: Imagine two separate spreadsheets — one with stock data, one with economic data. We paste them together side by side, matched by date.*

Calculating excess returns

```
data[symbols] = data[symbols].sub(data["RF"], axis = 0)
```

- ✓ `data[symbols]` — selects only the 15 stock return columns.
- ✓ `.sub(data["RF"], axis = 0)` — subtracts the daily RF (risk-free rate) from every stock return, row by row.
- ✓ Why? In finance, we care about returns ABOVE the risk-free rate. If a stock returns 1% but a risk-free Treasury also returns 1%, the stock added no value. Excess return = Stock Return - RF.
- ✓ After this step, each stock column contains excess returns (the dependent variable Y in our regression).

■ *Analogy: If a savings account pays 2% and your stock also pays 2%, your stock earned nothing extra. The 'bonus' above the safe rate is what we're measuring.*

Creating the AAPL-only dataset (showcase)

```
filter_list = fivefactors + ["AAPL"]  
aapl = data[filter_list]  
aapl
```

- ✓ `filter_list = fivefactors + ["AAPL"]` — creates a list combining the 5 factor names with 'AAPL', giving us 6 columns to keep.
- ✓ `data[filter_list]` — filters the full dataset down to just those 6 columns.
- ✓ `aapl` — the resulting DataFrame is our showcase dataset used to demonstrate EDA and regression before running it on all 15 stocks.

■ *Analogy: Before cooking a recipe for 15 people, you make a test batch for 1 person first to make sure it works.*

6 Exploratory Data Analysis (EDA) for AAPL

Before building a regression model, we explore the data visually and statistically. This helps us understand relationships between variables and confirm our assumptions.

Calculating the correlation matrix

```
corr_matrix = aapl.corr()
```

- ✓ `aapl.corr()` — calculates the pairwise correlation between every pair of columns in the AAPL dataset.
- ✓ Correlation ranges from -1 to +1: +1 means they move perfectly together, -1 means they move perfectly opposite, 0 means no relationship.
- ✓ We save it as `corr_matrix` to use in the heatmap next.

■ *Analogy: Like checking whether students who study more also score higher — and by how much.*

Visualising correlations with a heatmap

```
plt.figure(figsize=(12,9))  
sns.heatmap(corr_matrix, vmax=0.9, square=True, annot=True, cmap='RdBu')  
plt.show()
```

- ✓ `plt.figure(figsize=(12,9))` — creates a blank canvas for the chart (12 wide, 9 tall).
- ✓ `sns.heatmap()` — draws a colour-coded grid of all pairwise correlations.
- ✓ `vmax=0.9` — sets the maximum colour intensity at 0.9 correlation.
- ✓ `square=True` — makes each cell a square (visually cleaner).
- ✓ `annot=True` — prints the actual correlation numbers inside each cell.
- ✓ `cmap='RdBu'` — colours strong positive correlations blue, strong negative correlations red, near-zero correlations white.
- ✓ Key insight: look for which factor has the strongest (darkest blue) correlation with AAPL — that tells us the most important predictor.

Creating pairwise scatterplots

```
sns.pairplot(aapl, height=2)  
plt.show()
```

- ✓ `sns.pairplot()` — draws a grid of small scatterplots showing every variable plotted against every other variable.
- ✓ `height=2` — each small subplot is 2 inches tall.
- ✓ Diagonal cells show the distribution (histogram) of each variable.
- ✓ Off-diagonal cells show scatterplots — if points form a visible diagonal line, those two variables are correlated.
- ✓ We're looking for a clear linear relationship between AAPL and `Mkt_Prem`.

■ *Analogy: Like a team photo grid — every player photographed with every other player. You quickly spot who tends to stand together.*

Confirming linear relationship with Implot

```
sns.lmplot(x = "Mkt_Prem", y = "AAPL", data = aapl)
plt.show()
```

- ✓ `sns.lmplot()` — draws a scatterplot with a fitted regression line through the data points.
- ✓ `x = "Mkt_Prem"` — puts the Market Risk Premium on the horizontal axis.
- ✓ `y = "AAPL"` — puts the AAPL excess return on the vertical axis.
- ✓ The shaded band around the line is the confidence interval.
- ✓ A tight cluster of points along an upward-sloping line confirms a strong, positive linear relationship — meaning when markets go up, AAPL tends to go up proportionally.

7 Building the Regression Model for AAPL

We now define and fit the first regression model — for AAPL. This is our showcase model. The goal is to find the best-fit mathematical equation that explains AAPL's excess returns using the five Fama/French factors.

Defining the regression model

```
model = smf.ols("AAPL ~ Mkt_Prem + SMB + HML + RMW + CMA", data = aapl)
```

- ✓ `smf.ols()` — stands for Ordinary Least Squares. This is the standard method for fitting a linear regression.
- ✓ `"AAPL ~ Mkt_Prem + SMB + HML + RMW + CMA"` — this is the regression formula written in plain language as a string.
- ✓ The `~` symbol means 'is explained by'. Left side is Y (what we want to predict), right side lists the X variables (the predictors).
- ✓ Statsmodels automatically adds an intercept (constant) to the model.
- ✓ `data = aapl` — tells the model which DataFrame to find the variables in.
- ✓ `model` — at this stage, we've only defined the model structure. It hasn't been fitted to data yet.

■ *Analogy: Like writing out a recipe before you start cooking. You've listed the ingredients but haven't mixed anything yet.*

Fitting the model to the data

```
results = model.fit()
```

- ✓ `model.fit()` — this is where the actual maths happens. The OLS algorithm finds the coefficient values that minimise the total prediction error across all data points.
- ✓ `results` — stores everything about the fitted model: coefficients, p-values, R-squared, residuals, and more.

■ *Analogy: Now you actually cook the recipe. The oven (algorithm) does its work and produces the finished result.*

Viewing the full regression summary

```
print(results.summary())
```

- ✓ `results.summary()` — prints a detailed statistical report of the regression.
- ✓ Key things to look at in the summary:
 - ✓ • R-squared: how much of the variation in AAPL returns the model explains (higher = better).
 - ✓ • coef column: the regression coefficients — how much each factor moves AAPL per 1% factor move.
 - ✓ • P>|t| column: p-values — whether each factor is statistically significant.
 - ✓ • Durbin-Watson statistic: whether residuals show autocorrelation.

Checking which factors are significant

```
significance_level = 0.05  
results.pvalues < significance_level
```

- ✓ `significance_level = 0.05` — sets our threshold at 5%. This is the standard cut-off in statistics.
- ✓ `results.pvalues` — retrieves the p-value for each coefficient. The p-value tells us: 'if this factor had NO effect, how likely is it we'd see results this extreme by chance?'
- ✓ `< significance_level` — compares each p-value to 0.05. Returns True if the factor IS significant, False if it is NOT.
- ✓ A p-value below 0.05 means we have enough evidence (at 95% confidence) that the factor genuinely explains AAPL's return.

■ *Analogy: Imagine flipping a coin 100 times and getting 70 heads. A p-value tells you how surprising that result is if the coin were fair. If it's very surprising ($p < 0.05$), you conclude the coin is biased.*

Extracting the coefficients

```
results.params
```

- ✓ `results.params` — returns the regression coefficients (the 'slopes') for each factor plus the intercept.
- ✓ Each coefficient tells you: 'for every 1% increase in this factor, AAPL's excess return changes by this many percentage points, holding all other factors constant.'
- ✓ The intercept (called 'Intercept') is the alpha — the portion of AAPL's return not explained by any of the five factors.

Getting the R-squared

```
results.rsquared
```

- ✓ `results.rsquared` — returns the coefficient of determination (R-squared) as a single number between 0 and 1.
- ✓ `R-squared = 0.65` means the five factors together explain 65% of the day-to-day variation in AAPL's excess returns.
- ✓ The remaining 35% is unexplained — due to company-specific news, random events, etc.

■ *Analogy: If R-squared is 0.65, think of it like weather forecasting: the model correctly explains 65% of what happened, but 35% was unpredictable noise.*

8 Testing for Autocorrelation (Durbin-Watson Test)

Autocorrelation (also called serial correlation) is a common problem in time series regression. It occurs when today's prediction error is related to yesterday's — meaning the model is missing a pattern. If autocorrelation is present, our statistical tests become unreliable. We check for it visually and then with the Durbin-Watson test.

Plotting the regression residuals

```
results.resid.plot(figsize = (12, 8))  
plt.show()
```

- ✓ `results.resid` — the residuals are the differences between what the model predicted and what actually happened (actual - predicted).
- ✓ `.plot(figsize = (12, 8))` — draws the residuals over time as a line chart.
- ✓ What to look for: if residuals bounce randomly above and below zero with no pattern, autocorrelation is unlikely. If you see waves or drifts, it may be present.

■ *Analogy: After estimating your commute time each day, residuals are how wrong you were. If you're always late on Mondays specifically, that's a pattern (autocorrelation). If your errors are random, you're fine.*

Calculating the Durbin-Watson statistic

```
from statsmodels.stats.stattools import durbin_watson  
dw = durbin_watson(results.resid)  
dw
```

- ✓ `from statsmodels.stats.stattools import durbin_watson` — imports the specific function we need from Statsmodels.
- ✓ `durbin_watson(results.resid)` — calculates the DW statistic from the model's residuals.
- ✓ DW always falls between 0 and 4. The key interpretations are:
 - ✓ • $DW \approx 2$: no autocorrelation (ideal)
 - ✓ • $DW < 1.5$: positive autocorrelation (a problem)
 - ✓ • $DW > 2.5$: negative autocorrelation (also a problem)
- ✓ `dw` — displays the computed statistic.

Interpreting the Durbin-Watson result

```
dw > 1.6 and dw < 2.4
```

- ✓ This line checks whether the DW statistic falls within the acceptable range [1.6, 2.4].
- ✓ If this returns `True` — no significant autocorrelation. Our model is clean and the regression results are valid.
- ✓ If it returned `False` — we would need to correct the model (e.g. using Newey-West standard errors).
- ✓ Result for AAPL: `True` — the model passes the autocorrelation test.

9 Running Regression for All 15 Stocks (The Loop)

Now we scale up from AAPL to all 15 stocks. Instead of writing 15 separate models manually, we use a loop — Python repeats the same steps automatically for each stock and saves the results.

Creating empty results tables

```
coeff = pd.DataFrame(columns = ['Intercept', 'Mkt_Prem', 'SMB', 'HML', 'RMW', 'CMA'])
signif = pd.DataFrame(columns = ['Intercept', 'Mkt_Prem', 'SMB', 'HML', 'RMW', 'CMA'])
```

- ✓ `pd.DataFrame(columns = [...])` — creates an empty table with the specified column headers but no rows yet.
- ✓ `coeff` — will store the regression coefficient values (numbers) for all 15 stocks — one row per stock.
- ✓ `signif` — will store True/False significance flags for all 15 stocks — one row per stock.
- ✓ Think of these as two blank answer sheets that we'll fill in as the loop runs.

The regression loop — running all 15 models

```
for symbol in symbols:
    df = data[fivefactors + [symbol]]
    model = smf.ols('{ } ~ Mkt_Prem + SMB + HML + RMW + CMA'.format(symbol), data = df)
    results = model.fit()
    coeff.loc[symbol] = results.params
    signif.loc[symbol] = results.pvalues < significance_level
    signif.loc[symbol, 'rsquared'] = results.rsquared
coeff
```

- ✓ `for symbol in symbols:` — starts a loop. Python will repeat everything indented below for each of the 15 stock tickers in the `symbols` list.
- ✓ `df = data[fivefactors + [symbol]]` — for the current stock, extract a mini-table with just the 5 factors and that stock's excess return column.
- ✓ `smf.ols('{ } ~ ...'.format(symbol), data = df)` — builds the regression formula dynamically, inserting the current stock's name where `{ }` appears. For AAPL it becomes `'AAPL ~ Mkt_Prem + ...'`
- ✓ `results = model.fit()` — fits the OLS model to the data for this stock.
- ✓ `coeff.loc[symbol] = results.params` — saves the 6 coefficients into the `coeff` table, in the row named after the current stock.
- ✓ `signif.loc[symbol] = results.pvalues < significance_level` — saves True/False significance for each coefficient into the `signif` table.
- ✓ `signif.loc[symbol, 'rsquared'] = results.rsquared` — also saves the R-squared value in a new column in `signif`.
- ✓ `coeff` — after the loop finishes, displays the completed coefficients table (Table 1 for the fund management team).

■ *Analogy: Like a photocopier running the same document 15 times, each time swapping out one stock name. You set up the process once and let it run automatically.*

Displaying the significance table

```
signif
```

- ✓ After running the loop a second time to populate signif, typing signif displays the completed significance table.
- ✓ Each cell shows True (factor is significant for that stock) or False (not significant).
- ✓ The rsquared column shows how well the 5-factor model explains each stock's returns.
- ✓ This is Table 2 for the fund management team.

10 Ranking Factor Importance & Model Quality

Finally, we summarise which factors matter most across all 15 stocks, and how well the overall model performs. This is the key strategic insight for the fund management team.

Ranking factors by significance

```
signif[fivefactors].mean().sort_values(ascending = False).mul(100)
```

- ✓ `signif[fivefactors]` — selects only the 5 factor columns from the significance table (ignoring the Intercept and `rsquared` columns).
- ✓ `.mean()` — for each factor, calculates the average of the True/False values across all 15 stocks. Since True=1 and False=0, the mean gives the proportion of stocks for which the factor is significant.
- ✓ `.sort_values(ascending = False)` — sorts factors from most significant (highest proportion) to least significant.
- ✓ `.mul(100)` — multiplies by 100 to express as a percentage (e.g. 1.0 becomes 100%, meaning significant for all 15 stocks).
- ✓ Result: `Mkt_Prem` = 100% (always significant), followed by `SMB`, `HML`, `RMW`, and `CMA` in descending order.

■ *Analogy: Like asking: 'Which teacher's absence most disrupts the school?' If 100% of classes are affected when the maths teacher is away, maths is the most important factor.*

Calculating average model R-squared

```
signif.rsquared.mean()
```

- ✓ `signif.rsquared` — selects just the `rsquared` column from the `signif` table (which stores the R-squared for each of the 15 models).
- ✓ `.mean()` — calculates the average R-squared across all 15 stocks.
- ✓ Result: the average R-squared exceeds 0.50 — meaning the Fama/French 5 Factor Model explains more than 50% of each stock's daily return variation on average.
- ✓ This is a strong result and validates the 5-factor approach for the fund's equity universe.

■ *Analogy: Like averaging a student's test scores across 15 subjects. An average above 50% means the study method (5 factors) is broadly effective.*

Quick Reference — Key Python Methods Used

Method / Function	What it does
<code>pd.read_csv()</code>	Loads a CSV file into a Pandas DataFrame (table)
<code>df.isnull().sum()</code>	Counts missing values per column
<code>df.info()</code>	Shows data types, non-null counts, and memory usage
<code>df.pct_change().mul(100)</code>	Converts prices to percentage returns
<code>df.dropna()</code>	Removes rows that contain any missing values
<code>pd.concat()</code>	Joins two DataFrames side by side or top to bottom

<code>df.sub(other, axis=0)</code>	Subtracts another Series row by row
<code>df.corr()</code>	Calculates pairwise correlation matrix
<code>sns.heatmap()</code>	Draws a colour-coded correlation grid
<code>sns.pairplot()</code>	Draws scatterplots for every pair of variables
<code>sns.lmplot()</code>	Draws a scatterplot with a fitted regression line
<code>smf.ols(formula, data)</code>	Defines an OLS linear regression model
<code>model.fit()</code>	Fits the model — computes the best coefficients
<code>results.summary()</code>	Prints the full regression output table
<code>results.params</code>	Returns the regression coefficients
<code>results.pvalues</code>	Returns the p-values for each coefficient
<code>results.rsquared</code>	Returns the R-squared (model fit quality)
<code>results.resid</code>	Returns the residuals (actual minus predicted)
<code>durbin_watson(resid)</code>	Calculates the Durbin-Watson autocorrelation statistic
<code>df.mean().sort_values()</code>	Calculates column averages and sorts them